GUÍA DE ESTUDIO DE JAVASCRIPT

Valores y variables · Operadores · Estructuras de control · Funciones · Objetos y JSON · Arrays y sus métodos

Esta guía junta en un solo documento tus dos hojas de repaso (la de los tres PDF de fundamentos y la de arrays). Está ordenada de lo más básico a lo más avanzado: primero cómo se declaran y convierten los valores, luego los operadores y las estructuras de control, después las funciones y, al final, los arrays con todos sus métodos, que suelen ser la parte donde más se falla en un examen.

Cómo estudiarla paso a paso. Primero lee las explicaciones en texto de cada apartado; después mira los ejemplos y tápalos para intentar adivinar el resultado antes de leerlo; por último haz el mini-test del final sin mirar las soluciones. En los métodos de arrays no memorices solo el nombre: fíjate siempre en tres cosas, **qué hace**, **qué devuelve** y **si modifica el array original**.

Leyenda de colores. En las tablas de arrays, Sí en rojo significa que el método MODIFICA el array original (mutación) y No en verde significa que deja el original intacto y devuelve algo nuevo. Los recuadros azules son ideas clave, los verdes son trucos para recordar y los rojos son errores típicos.

Índice

1	Variables, tipos y conversiones
2	Operadores
3	Estructuras de control y errores
4	Funciones y programación funcional
5	Objetos, JSON y desestructuración
6	Arrays: creación, diagrama y todos los métodos
7	map, filter y reduce: ejemplo visual paso a paso
8	Diferencias que se confunden
9	Trampas típicas de examen (con resultado)
10	Repaso rápido: «si te piden…, piensa en…»
11	Mini-test de práctica y soluciones

1. Variables, tipos y conversiones

En JavaScript guardas datos en variables. Hoy se usan casi siempre dos palabras: `const` para lo que no va a recibir otra asignación y `let` para lo que sí va a cambiar. `var` es la forma antigua y el material del profesor recomienda no usarla. Ojo: con `const` no puedes reasignar la variable, pero si guarda un array u objeto sí puedes cambiar su contenido (`const a = []; a.push(1)` es válido).

Palabra	Qué hace	Ejemplo
let	Declara una variable cuyo valor puede cambiar.	`let edad = 18; edad = 19`
const	Declara una variable que no puede recibir otra asignación.	`const PI = 3.14`
var	Forma antigua. Se evita porque su ámbito y su hoisting dan sorpresas.	`var edad = 18`

Tipos de valores y typeof

El operador `typeof` devuelve un texto con el tipo del valor. Los tipos básicos son número, cadena, booleano, `undefined`, `null`, objeto y función. Dos casos que se preguntan mucho: `typeof null` da `"object"` (es un fallo histórico del lenguaje) y `typeof []` también da `"object"`, por eso para saber si algo es un array se usa `Array.isArray()`.

Expresión	Resultado
`typeof 42`	`"number"`
`typeof "hola"`	`"string"`
`typeof true`	`"boolean"`
`typeof undefined`	`"undefined"`
`typeof null`	`"object"` (caso especial)
`typeof [1,2]`	`"object"` (usa `Array.isArray([1,2])` -> `true`)
`typeof function(){}`	`"function"`

Funciones y métodos básicos (conversión, números y cadenas)

Nombre	Para qué sirve	Ejemplo
Salida y comprobación		
console.log()	Muestra información en la consola.	`console.log("Hola")`
window.alert()	Muestra un mensaje en una ventana del navegador.	`window.alert("Hola")`
typeof	Indica el tipo de un valor.	`typeof 42` -> `"number"`
Array.isArray()	Comprueba si un valor es un array.	`Array.isArray([1,2])` -> `true`
Object.is()	Compara dos valores con reglas parecidas a `===` (y `NaN` es igual a `NaN`).	`Object.is(NaN, NaN)` -> `true`
Conversión entre tipos		
parseInt()	Cadena -> número entero. Lee hasta donde puede; si no hay número da `NaN`.	`parseInt("25")` -> 25 · `parseInt("25px")` -> 25
parseFloat()	Cadena -> número decimal.	`parseFloat("3.14")` -> 3.14
toString()	Número -> cadena.	`(25).toString()` -> `"25"`
JSON.stringify()	Objeto JavaScript -> texto JSON.	`JSON.stringify({nombre:"Ana"})`
JSON.parse()	Texto JSON -> objeto JavaScript.	`JSON.parse('{"edad":18}')`
Math		
Math.trunc()	Elimina la parte decimal (sin redondear).	`Math.trunc(4.8)` -> 4 · `Math.trunc(-4.8)` -> -4
Math.round()	Redondea al entero más cercano.	`Math.round(4.6)` -> 5
Math.floor()	Redondea hacia abajo.	`Math.floor(4.9)` -> 4 · `Math.floor(-4.8)` -> -5
Math.max()	Mayor de varios números (con spread para arrays).	`Math.max(3, 8, 5)` -> 8 · `Math.max(...nums)`
Math.abs()	Valor absoluto.	`Math.abs(-7)` -> 7
Math.random()	Número aleatorio entre 0 (incluido) y 1 (excluido).	`Math.floor(Math.random() * 10)` -> 0 a 9
Cadenas (strings)		
substring()	Extrae una parte indicando inicio y fin (el fin no se incluye).	`"Hola".substring(0, 2)` -> `"Ho"`
trim()	Quita espacios al principio y al final.	`" Hola ".trim()` -> `"Hola"`
toUpperCase()	Pasa a mayúsculas.	`"hola".toUpperCase()` -> `"HOLA"`
toLowerCase()	Pasa a minúsculas.	`"HOLA".toLowerCase()` -> `"hola"`
split() / join()	`split` divide una cadena en array; `join` hace lo contrario.	`"a,b".split(",")` -> `["a","b"]`
length	Longitud de una cadena o array.	`"Hola".length` -> 4
Template literal	Cadena con `${}` y varias líneas, entre acentos graves.	`Hola ${nombre}`

2. Operadores

Los operadores se entienden mejor por familias. Lo más importante para el examen es distinguir la igualdad estricta de la no estricta, saber que `+` también concatena y no confundir `??` con `||`.

Operador	Para qué sirve	Ejemplo
Aritméticos		
+	Suma números o concatena cadenas.	`5 + 3` -> 8 · `"Hola " + "Ana"` · `"5" + 3` -> `"53"`
- * /	Resta, multiplica, divide.	`8 - 3` -> 5 · `4 * 3` -> 12 · `10 / 2` -> 5
%	Resto de una división (sirve para saber si un número es par).	`10 % 3` -> 1 · `n % 2 === 0`
****	Potencia.	`2 ** 3` -> 8

Operador	Para qué sirve	Ejemplo
++ --	Suman o restan 1.	`x++` · `x--`
Asignación		
=	Asigna un valor.	`x = 5`
+= -= *= /=	Operan y asignan a la vez.	`x += 2` equivale a `x = x + 2`
Comparación		
< <= > >=	Menor, menor o igual, mayor, mayor o igual.	`3 < 5` · `5 >= 5`
=== !==	Igualdad / desigualdad ESTRICTA: comparan valor y tipo. Es lo recomendable.	`5 === "5"` -> false · `5 !== "5"` -> true
== !=	Igualdad / desigualdad no estricta: pueden convertir tipos.	`5 == "5"` -> true
Lógicos		
&&	AND: deben cumplirse las dos condiciones.	`edad >= 18 && activo`
||	OR: basta con que se cumpla una (devuelve el primer valor «verdadero»).	`edad >= 18 || permiso`
!	NOT: invierte el booleano.	`!true` -> false
Especiales		
??	Valor alternativo SOLO si lo de la izquierda es `null` o `undefined`.	`nombre ?? "Desconocido"` · `0 ?? 5` -> 0
?.	Acceso seguro: si el objeto es null/undefined no da error, devuelve undefined.	`persona?.nombre`
?:	Ternario: condición corta con dos resultados.	`edad >= 18 ? "Mayor" : "Menor"`
...	Spread: expande los elementos de un iterable.	`Math.max(...numeros)` · `[0, ...numeros]`
delete	Elimina una propiedad de un objeto.	`delete persona.edad`
instanceof	Comprueba si un objeto pertenece a un tipo.	`x instanceof String`
in	Comprueba si existe una propiedad (y aparece en `for...in`).	`"nombre" in persona`

> **Truco para recordar `??` y `||`.** `||` mira si el valor es «falso» (0, "", false, null, undefined, NaN) y entonces usa el alternativo; `??` solo mira si es null o undefined. Por eso `0 || "x"` da `"x"` pero `0 ?? "x"` da `0`. Usa `??` cuando el 0 o la cadena vacía sean valores válidos.

3. Estructuras de control y errores

Las estructuras de control deciden qué código se ejecuta y cuántas veces. Se dividen en tres grupos: **condicionales** (`if`, `switch`), **bucles** (`for`, `while`, ...) y **gestión de errores** (`try`, `catch`, `finally`, `throw`).

Estructura	Para qué sirve	Ejemplo
Condicionales		
if / else if / else	Ejecuta código según se cumpla o no una condición.	`if (x > 0) {...} else if (x === 0) {...} else {...}`
switch / case / default	Elige entre varios casos. `default` se ejecuta si ningún `case` coincide.	`switch (op) { case 1: ... break; default: ... }`
Bucles		
for	Bucle con inicialización, condición y actualización. Ideal cuando sabes cuántas vueltas.	`for (let i = 0; i < 5; i++)`
while	Repite MIENTRAS la condición sea verdadera (puede no ejecutarse ninguna vez).	`while (x < 10) { x++ }`
do...while	Ejecuta el bloque al menos UNA vez y luego comprueba.	`do { x++ } while (x < 10)`
for...of	Recorre los VALORES de un iterable (array, cadena).	`for (const x of numeros)`
for...in	Recorre las CLAVES (propiedades) de un objeto.	`for (const clave in persona)`

Estructura	Para qué sirve	Ejemplo
break	Sale del bucle o del `switch`.	`if (encontrado) break`
continue	Salta el resto de esta vuelta y pasa a la siguiente.	`if (x < 0) continue`
Errores		
try	Contiene el código que puede lanzar una excepción.	`try { JSON.parse(texto) }`
catch	Captura la excepción producida en `try`.	`catch (e) { console.log(e) }`
finally	Se ejecuta SIEMPRE, haya habido error o no.	`finally { cerrar() }`
throw	Lanza una excepción manualmente.	`throw Error("No encontrado")`
Otros		
'**use strict**'	Activa el modo estricto: prohíbe cosas problemáticas (por ejemplo usar una variable sin declarar).	`'use strict'`

Ejemplos para ver la diferencia

```
// switch: sin break, el código "se cae" al siguiente case
const dia = 2;
switch (dia) {
  case 1: console.log("Lunes"); break;
  case 2: console.log("Martes"); break;     // imprime "Martes" y sale
  default: console.log("Otro día");
}

// do...while: se ejecuta una vez aunque la condición sea falsa
let x = 10;
do { console.log(x); x++; } while (x < 5);   // imprime 10

// for...of (valores) frente a for...in (claves)
for (const n of [10, 20]) console.log(n);          // 10, 20
for (const k in { a: 1, b: 2 }) console.log(k);    // "a", "b"

// try / catch / finally
try {
  JSON.parse("{mal}");                // lanza SyntaxError
} catch (e) {
  console.log("JSON inválido");       // se ejecuta
} finally {
  console.log("esto sale siempre");   // se ejecuta
}
```

Elegir el bucle (paso a paso). 1) ¿Sé cuántas vueltas? Usa `for`. 2) ¿Recorro un array y quiero sus valores? Usa `for...of`. 3) ¿Recorro las propiedades de un objeto? Usa `for...in`. 4) ¿Repito hasta que ocurra algo? Usa `while`, y `do...while` si debe ejecutarse al menos una vez. Con arrays evita `for...in`: recorre los índices como texto (`"0"`, `"1"`...).

4. Funciones y programación funcional

Una función agrupa código reutilizable. Se **declara** una vez y se **llama** las veces que haga falta pasándole argumentos. En JavaScript las funciones son valores: se pueden guardar en variables, pasar a otras funciones (callbacks) o devolver desde otras funciones. Cuando una función recibe o devuelve otra función se llama **de orden superior** (`map`, `filter`, `forEach`, `reduce`... lo son).

Concepto	Para qué sirve	Ejemplo
Función normal	Agrupa código reutilizable.	`function sumar(a, b) { return a + b }`
Llamar	Ejecuta la función pasando argumentos.	`sumar(2, 3)` -> 5
Parámetro / argumento	Parámetro = variable de la definición; argumento = valor al llamar.	`function saludar(nombre)` / `saludar("Ana")`
return	Devuelve el resultado y TERMINA la función (lo de debajo no se ejecuta).	`return a + b`

Concepto	Para qué sirve	Ejemplo
Función anónima	Función sin nombre, útil cuando se usa una sola vez.	`function(x) { return x * 2 }`
Función flecha	Forma corta. Si es una sola expresión, el `return` es implícito.	`x => x * 2` · `(a, b) => a + b`
Orden superior	Recibe una función o devuelve una función.	`[1,2,3].map(x => x * 2)`
Callback	Función que se pasa como argumento a otra.	`.map(x => x * 2)` (la flecha es el callback)
Parámetro predeterminado	Valor por defecto si no se pasa argumento.	`(x, y = 0) => x + y`
Rest ...	Recoge varios argumentos en un array (en la DEFINICIÓN).	`(...numeros) => numeros.length`
Spread ...	Expande un array en valores separados (en la LLAMADA).	`Math.max(...numeros)`
Hoisting	Las declaraciones se «elevan» al inicio de su ámbito (ver recuadro).	`saludar(); function saludar() {}` funciona
Modo estricto	Limita características problemáticas.	`'use strict'`

La misma función escrita de cinco formas

```
function doble(x) { return x * 2 }          // 1) declaración normal
const doble2 = function (x) { return x * 2 } // 2) anónima guardada en una variable
const doble3 = (x) => { return x * 2 }       // 3) flecha con llaves (hay que poner return)
const doble4 = x => x * 2                    // 4) flecha corta (return implícito)
const suma = (...nums) => nums.reduce((t, n) => t + n, 0)   // 5) con rest
suma(1, 2, 3)     // 6

// Si la flecha devuelve un objeto literal, va entre paréntesis:
const crear = nombre => ({ nombre: nombre })
```

Hoisting (lo que cae en examen). Las funciones declaradas con `function` se pueden usar ANTES de escribirlas. Con `var`, la variable existe pero vale `undefined` hasta que se asigna. Con `let` y `const` usarlas antes de declararlas da `ReferenceError`. Y en modo estricto, asignar a una variable sin declarar también da `ReferenceError`.

Rest frente a spread: el mismo símbolo, dos usos

```
function sumar(...nums) { return nums.length }   // REST: junta argumentos en un array
sumar(1, 2, 3)                                   // 3

const numeros = [4, 9, 2];
Math.max(...numeros)                             // SPREAD: separa el array en valores -> 9
const copia = [0, ...numeros]                    // [0, 4, 9, 2]
```

5. Objetos, JSON y desestructuración

Un objeto guarda pares clave-valor. Una matriz (array) es un tipo especial de objeto con índices numéricos y la propiedad `length`. JSON es un formato de texto para intercambiar datos: `JSON.stringify` convierte un objeto en texto y `JSON.parse` hace el camino contrario. Si el texto no es JSON válido, `JSON.parse` lanza un error, por eso se suele envolver en `try / catch`.

Elemento	Para qué sirve	Ejemplo
objeto.propiedad	Accede a una propiedad con punto.	`persona.nombre`
objeto["propiedad"]	Accede con corchetes (útil si la clave está en una variable).	`persona["nombre"]`
{} / []	Crea un objeto literal / un array literal.	`{nombre: "Ana", edad: 18}` · `[1, 2, 3]`
delete	Elimina una propiedad.	`delete persona.edad`
{...objeto}	Spread de objeto: copia (superficial) sus propiedades.	`const copia = {...persona}`

Elemento	Para qué sirve	Ejemplo
[...array]	Spread de array: copia o amplía un array.	`[0, ...numeros]`
JSON.stringify()	Objeto -> texto JSON.	`JSON.stringify(persona)` -> `'{"nombre":"Ana"}'`
JSON.parse()	Texto JSON -> objeto.	`JSON.parse(texto)`

Desestructuración paso a paso

Desestructurar es sacar valores de un array u objeto directamente a variables. En arrays se hace **por posición**; en objetos **por nombre**. Se puede anidar, usar valores predeterminados y recoger lo que sobra con `...rest`.

```
// ARRAY: por posición
const [a, b] = [10, 20];              // a = 10, b = 20
const [x, , z, ...resto] = [1, 2, 3, 4, 5];   // x=1, (se salta el 2), z=3, resto=[4, 5]

// OBJETO: por nombre
const { nombre, edad = 18 } = { nombre: "Ana" };  // nombre="Ana", edad=18 (valor predeterminado)

// ANIDADA + rest de objeto
const { n1, dir: { ciudad }, ...otros } =
  { n1: "Ana", dir: { ciudad: "Oviedo" }, edad: 20, rol: "x" };
// n1 = "Ana", ciudad = "Oviedo", otros = { edad: 20, rol: "x" }
```

Errores típicos. 1) En objetos importa el NOMBRE, no el orden. 2) El valor predeterminado solo se usa si llega `undefined` (no si llega `null` ni `0`). 3) `{...obj}` y `[...arr]` hacen copias superficiales: si dentro hay otros objetos o arrays, se siguen compartiendo.

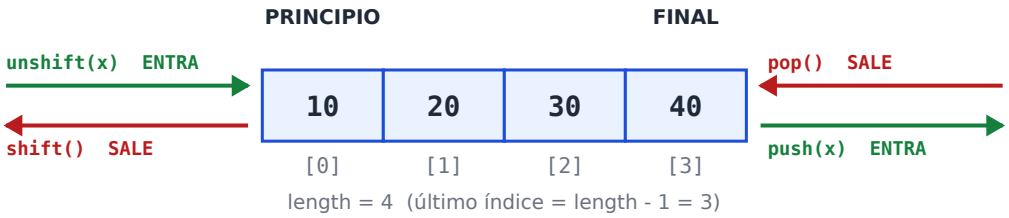
6. Arrays: creación, diagrama y todos los métodos

Un array guarda una lista ordenada de elementos. Los índices empiezan en 0 y el último elemento está en `length - 1`. Lo normal es crearlo con corchetes (`[]`); el constructor `new Array()` tiene un detalle raro: con un solo número crea un array VACÍO de esa longitud, y con varios valores crea el array con ellos.

```
const numeros = [1, 2, 3];          // forma habitual (notación literal)
const nombres = new Array("Juan", "Ana");   // constructor -> ["Juan", "Ana"]
new Array(3)      // [ <3 elementos vacíos> ]  (¡longitud 3, sin valores!)
new Array(3, 4)   // [3, 4]

numeros[0]        // 1      (acceso por índice)
numeros[1] = 20   // modifica -> [1, 20, 3]
numeros.length    // 3
```

Diagrama: añadir y quitar por los dos extremos



Regla mnemotécnica. `push` / `pop` trabajan por el FINAL (los más usados, como una pila de platos). `unshift` / `shift` trabajan por el PRINCIPIO. Los cuatro modifican el array original. `push` y `unshift` devuelven la nueva longitud; `pop` y `shift` devuelven el elemento que han quitado.

Tabla completa de métodos, por familias

Lee cada fila de izquierda a derecha: nombre, qué hace, si toca el original, qué te devuelve y un ejemplo. La columna «Modifica» (roja = sí, verde = no) es la que más conviene tener clara.

Método	Qué hace	Modifica	Devuelve	Ejemplo
A) Añadir y quitar elementos				
push(...x)	Añade uno o varios elementos al final.	**Sí**	La nueva longitud	`let a=[1,2]; a.push(3)` -> a = [1,2,3]
pop()	Elimina el último elemento.	**Sí**	El elemento eliminado	`[1,2,3].pop()` -> 3
unshift(...x)	Añade al principio.	**Sí**	La nueva longitud	`let a=[2,3]; a.unshift(1)` -> [1,2,3]
shift()	Elimina el primero.	**Sí**	El elemento eliminado	`[1,2,3].shift()` -> 1
splice(ini, n, ...x)	Inserta, borra o sustituye en cualquier posición.	**Sí**	Array con los elementos eliminados	`a.splice(1,1,10)` en [1,2,3] -> a = [1,10,3]
B) Copiar, cortar y unir				
slice(ini, fin)	Copia una parte (el índice `fin` NO se incluye). Admite negativos.	**No**	Un array nuevo	`[1,2,3,4].slice(1,3)` -> [2,3] · `.slice(-2)` -> [3,4]
concat(...)	Une arrays o valores.	**No**	Un array nuevo	`[1,2].concat([3,4],5)` -> [1,2,3,4,5]
C) Buscar				
indexOf(x)	Primera posición de un elemento.	**No**	Índice o -1	`["a","b"].indexOf("b")` -> 1

Método	Qué hace	Modifica	Devuelve	Ejemplo
lastIndexOf(x)	Última posición de un elemento.	**No**	Índice o -1	`["a","b","a"].lastIndexOf("a")` -> 2
includes(x)	Comprueba si existe.	**No**	true / false	`[1,2,3].includes(2)` -> true
find(fn)	Primer elemento que cumple la condición.	**No**	El elemento o undefined	`[2,4,7,8].find(n => n % 2 !== 0)` -> 7
findIndex(fn)	Como find, pero da la posición.	**No**	Índice o -1	`[2,4,7].findIndex(n => n > 5)` -> 2
findLastIndex(fn)	Busca desde el final.	**No**	Índice o -1	`[2,7,4,7].findLastIndex(n => n === 7)` -> 3
D) Recorrer y transformar				
forEach(fn)	Ejecuta una función por cada elemento («hago algo»).	**No**	undefined	`[1,2,3].forEach(n => console.log(n))`
map(fn)	Transforma cada elemento («transformo»).	**No**	Array nuevo, MISMA longitud	`[1,2,3].map(n => n * 2)` -> [2,4,6]
filter(fn)	Se queda con los que cumplen la condición («selecciono»).	**No**	Array nuevo (puede ser menor o vacío)	`[1,2,3,4].filter(n => n > 2)` -> [3,4]
E) Ordenar				
sort(fn)	Ordena. SIN función ordena como TEXTO.	**Sí**	El mismo array ordenado	`[10,2,5].sort((x,y) => x - y)` -> [2,5,10]
reverse()	Invierte el orden.	**Sí**	El mismo array invertido	`[1,2,3].reverse()` -> [3,2,1]
F) Convertir				
join(sep)	Une los elementos en una cadena (contrario de split).	**No**	Una cadena	`["Juan","Ana"].join(", ")` -> "Juan, Ana"
split(sep)	(De STRING) Divide un texto en un array.	**No (string)**	Un array	`"Juan,Ana".split(",")` -> ["Juan","Ana"]
toString()	Array a cadena separada por comas.	**No**	Una cadena	`[1,2,3].toString()` -> "1,2,3"
toLocaleString()	A cadena según la configuración regional.	**No**	Una cadena	`[1234567.89].toLocaleString("es-ES")` -> "1.234.567,89"
valueOf()	Devuelve el propio array.	**No**	El mismo array	`a.valueOf()` -> [1,2,3]
localeCompare() (string)	Compara cadenas respetando acentos; se usa dentro de sort.	**No (string)**	Número negativo, 0 o positivo	`["Ana","Álvaro"].sort((a,b) => a.localeCompare(b))`
G) Reducir a un único valor				
reduce(fn, inicial)	Combina todos los elementos en un solo resultado, de izquierda a derecha.	**No**	Un único valor	`[1,2,3,4].reduce((t,n) => t + n, 0)` -> 10
reduceRight(fn, inicial)	Igual, pero de derecha a izquierda.	**No**	Un único valor	`["a","b","c"].reduceRight((t,n) => t + n, "")` -> "cba"
H) Comprobar				
some(fn)	¿Al menos UNO cumple?	**No**	true / false	`[1,3,4].some(n => n % 2 === 0)` -> true
every(fn)	¿Cumplen TODOS?	**No**	true / false	`[2,4,6].every(n => n % 2 === 0)` -> true
Array.isArray(x)	¿Es realmente un array?	**No**	true / false	`Array.isArray([1,2])` -> true
I) Avanzados (ampliación)				
fill(v, ini, fin)	Rellena parte del array con un valor.	**Sí**	El mismo array	`[1,2,3,4].fill(0,1,3)` -> [1,0,0,4]
copyWithin(dest, ini, fin)	Copia elementos dentro del propio array, sobrescribiendo.	**Sí**	El mismo array	`[1,2,3,4].copyWithin(0,2,4)` -> [3,4,3,4]

Método	Qué hace	Modifica	Devuelve	Ejemplo
flat(prof)	Aplana arrays anidados.	**No**	Array nuevo	`[1,[2,3],[4]].flat()` -> [1,2,3,4]
flatMap(fn)	map + flat de un nivel.	**No**	Array nuevo	`[1,2,3].flatMap(n => [n, n*2])` -> [1,2,2,4,3,6]

Otros conceptos de arrays que aparecen en los PDF

Concepto	Qué hay que saber	Ejemplo
for / for...of	`for` recorre con índices; `for...of` recorre directamente los valores.	`for (const f of frutas) console.log(f)`
delete a[i]	Borra el valor pero deja un HUECO y NO reduce `length`. Para quitar de verdad usa `splice`.	`delete a[1]` en [1,2,3] -> [1, vacío, 3], length 3
Función de comparación	En `sort`, si devuelve negativo `a` va antes; si positivo `b` va antes.	`(x, y) => x - y` ascendente · `(x, y) => y - x` descendente
thisArg	Segundo parámetro opcional de `find`, `filter`, `map`... para fijar `this` dentro de la función.	`usuarios.filter(army.canJoin, army)`
Symbol.isConcatSpreadable	Hace que `concat()` trate un objeto parecido a un array como si lo fuera.	`obj[Symbol.isConcatSpreadable] = true`

splice paso a paso: sus tres usos

`splice(inicio, cuántosBorrar, ...nuevos)` es el método más confuso. Piensa en tres usos según qué pongas en el segundo parámetro: si borras y no añades, **eliminas**; si pones 0 y añades, **insertas**; si borras y añades, **sustituyes**. Siempre devuelve lo que ha eliminado y modifica el original.

```
let a = [1, 2, 3, 4, 5]; a.splice(1, 2);       // ELIMINAR 2 desde la pos. 1 -> a = [1, 4, 5]
let b = [1, 2, 3, 4, 5]; b.splice(1, 0, "x");  // INSERTAR sin borrar        -> b = [1, "x", 2, 3, 4, 5]
let c = [1, 2, 3, 4, 5]; c.splice(1, 1, "x");  // SUSTITUIR 1 elemento       -> c = [1, "x", 3, 4, 5]
```

sort: por qué [10, 2, 5].sort() sale mal

Sin función de comparación, `sort` convierte los elementos a texto y los ordena alfabéticamente, así que `"10"` va antes que `"2"`. Para números hay que pasar siempre una función de comparación. Para texto con acentos usa `localeCompare`.

```
[10, 2, 5, 1].sort()                    // [1, 10, 2, 5]   <- alfabético, ¡mal para números!
[10, 2, 5, 1].sort((a, b) => a - b)     // [1, 2, 5, 10]   ascendente
[10, 2, 5].sort((a, b) => b - a)        // [10, 5, 2]      descendente
["Ana","Álvaro","Zoe","Beto"].sort()    // ["Ana","Beto","Zoe","Álvaro"]  <- el acento va al final
["Ana","Álvaro","Zoe","Beto"].sort((a, b) => a.localeCompare(b))
                                        // ["Álvaro","Ana","Beto","Zoe"]
```

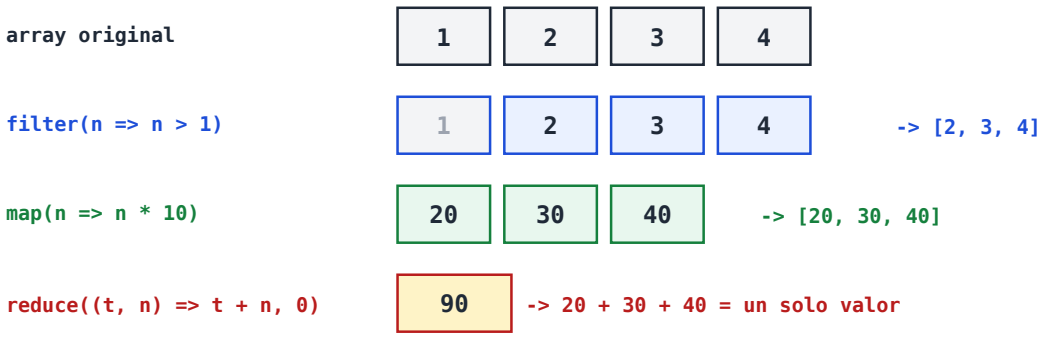
Qué métodos modifican el array original (para memorizar)

MODIFICAN el original (mutan)	NO modifican el original (devuelven algo nuevo)
`push pop shift unshift splice sort reverse fill copyWithin`	`slice concat map filter join flat flatMap find findIndex indexOf includes some every reduce forEach`

El PDF del profesor destaca tres: `splice`, `sort` y `reverse` modifican el original, mientras que `slice` no. En la lista roja he añadido el resto de mutadores para que no te pillen.

7. map, filter y reduce: ejemplo visual paso a paso

Estos tres métodos se encadenan a menudo (como en el ejemplo del profesor con `items.filter(...).map(...).join("")`). Cada uno recibe el resultado del anterior. Mira cómo viaja el array `[1, 2, 3, 4]` por la cadena:




```
const resultado = [1, 2, 3, 4]
  .filter(n => n > 1)            // [2, 3, 4]      selecciono
  .map(n => n * 10)              // [20, 30, 40]   transformo
  .reduce((t, n) => t + n, 0);   // 90             combino en un valor
```

Cómo funciona reduce, vuelta a vuelta

`reduce` mantiene un **acumulador** (`t`, el total). Empieza con el valor inicial (aquí 0) y en cada vuelta el callback recibe el acumulador y el elemento actual; lo que devuelve se convierte en el acumulador de la vuelta siguiente. Al final devuelve el acumulador.

Vuelta	t (acumulador)	n (elemento)	t + n (pasa a la siguiente vuelta)
1	0 (valor inicial)	20	20
2	20	30	50
3	50	40	90 <- resultado final

El canal del profesor, con un ejemplo que puedes probar

```
["  ana ", "", " luis"]
  .filter(s => s.trim() !== "")        // quita las cadenas vacías  -> ["  ana ", " luis"]
  .map(s => s.trim().toUpperCase())    // limpia y pasa a mayúsculas -> ["ANA", "LUIS"]
  .join(", ")                          // une en una cadena          -> "ANA, LUIS"
// (en el PDF se usan las funciones htmlEscape y enclose del profesor en lugar de trim/toUpperCase)
```

Idea para memorizar. `forEach` = hago algo con cada elemento (no devuelve nada) · `map` = transformo (mismo número de elementos) · `filter` = selecciono (igual o menos elementos) · `reduce` = combino todo en un único valor.

Dos fallos clásicos. 1) En `map` con llaves hay que escribir `return`: `[1,2,3].map(n => { n * 2 })` da `[undefined, undefined, undefined]`. 2) `reduce` sin valor inicial sobre un array vacío lanza un error; con valor inicial (`0`) devuelve ese valor. Pon siempre el valor inicial.

8. Diferencias que se confunden

Tema	Opción A	Opción B
Final / principio	`push()` añade al final · `pop()` quita del final	`unshift()` añade al principio · `shift()` quita del principio
Copiar / modificar	`slice()` crea otro array y NO modifica el original	`splice()` MODIFICA el original
Buscar	`find()` devuelve UN elemento (o undefined)	`filter()` devuelve un ARRAY con todos los que cumplen
Recorrer / transformar	`map()` crea un array nuevo transformado	`forEach()` ejecuta una función y no devuelve nada
Comprobar contenido	`indexOf()` devuelve la posición (o -1)	`includes()` devuelve true / false
Posición vs elemento	`find()` devuelve el elemento	`findIndex()` devuelve su posición

Tema	Opción A	Opción B
Reducir	`reduce()` de izquierda a derecha	`reduceRight()` de derecha a izquierda
Alguno / todos	`some()` basta con uno	`every()` deben cumplirlo todos
split / join	`split()` (de string) texto -> array	`join()` array -> texto
Igualdad	`===` compara valor y tipo	`==` convierte tipos (`5 == "5"` es true)
Valor alternativo	`??` solo si es null / undefined	`||` si es «falso» (0, "", false...)
Bucles de recorrido	`for...of` recorre VALORES (arrays)	`for...in` recorre CLAVES (objetos)
Rest / spread	Rest `...` en la definición: junta en un array	Spread `...` en la llamada: separa en valores
Declaración	`let` / `const`: recomendadas	`var`: antigua, se evita

9. Trampas típicas de examen (con resultado)

Todos estos resultados están comprobados ejecutando el código. Antes de mirar la columna de la derecha, intenta adivinar qué sale.

Código	Resultado	Por qué
`[10, 2, 5, 1].sort()`	`[1, 10, 2, 5]`	Sin función ordena como texto.
`typeof null` · `typeof []`	`"object"` · `"object"`	Para arrays usa `Array.isArray()`.
`0.1 + 0.2`	`0.30000000000000004`	Los decimales en binario no son exactos.
`"5" + 3` · `"5" - 3`	`"53"` · `2`	`+` concatena si hay una cadena; `-` convierte a número.
`0 || "x"` · `0 ?? "x"`	`"x"` · `0`	`||` trata el 0 como falso; `??` solo null/undefined.
`parseInt("25px")` · `parseInt("abc")`	`25` · `NaN`	Lee hasta donde puede; si no hay número, NaN.
`5 == "5"` · `5 === "5"`	`true` · `false`	`==` convierte tipos.
`{} === {}`	`false`	Los objetos se comparan por referencia, no por contenido.
`let y = x` **(x array);** `y.push(3)`	x también cambia	No es una copia: ambos apuntan al mismo array. Copia con `[...x]` o `x.slice()`.
`let a=[1,2,3]; delete a[1]`	`[1, vacío, 3]`, length 3	`delete` deja un hueco. Usa `splice`.
`[1,2,3].map(n => { n * 2 })`	`[undefined, undefined, undefined]`	Con llaves hace falta `return`.
`[].reduce((a,b) => a + b)`	TypeError	Sin valor inicial en array vacío. Usa `, 0`.
`[NaN].indexOf(NaN)` · `[NaN].includes(NaN)`	`-1` · `true`	`indexOf` usa `===`; `includes` sí encuentra NaN.
`new Array(3)` · `new Array(3, 4)`	3 huecos · `[3, 4]`	Un solo número = longitud.
`for (const i in ["a","b"])`	`"0"`, `"1"` (texto)	`for...in` da claves, no valores.
`[1,2,3,4,5].slice(-2)`	`[4, 5]`	Índice negativo = desde el final.
`Math.round(-2.5)` · `Math.trunc(-4.8)` · `Math.floor(-4.8)`	`-2` · `-4` · `-5`	Con negativos, `floor` baja y `trunc` solo corta.

10. Repaso rápido: «si te piden..., piensa en...»

Si te piden...	Piensa en...	Ejemplo
Valores y conversiones		
Mostrar algo en consola	`console.log()`	`console.log(nombre)`
Saber qué tipo tiene un valor	`typeof`	`typeof edad`
Saber si es un array	`Array.isArray()`	`Array.isArray(datos)`
Texto a entero / decimal	`parseInt()` / `parseFloat()`	`parseInt("20")` · `parseFloat("20.5")`

Si te piden...	Piensa en...	Ejemplo
Número a texto	`toString()`	`(20).toString()`
Objeto <-> JSON	`JSON.stringify()` / `JSON.parse()`	`JSON.stringify(persona)`
Operadores y control		
Comparar sin convertir tipos	`===` / `!==`	`edad === 18`
Combinar condiciones	`&&` `||` `!`	`edad >= 18 && activo`
Valor alternativo solo para null/undefined	`??`	`nombre ?? "Sin nombre"`
Acceso seguro a una propiedad	`?.`	`persona?.nombre`
Condición corta de 2 opciones	`?:`	`edad >= 18 ? "Mayor" : "Menor"`
Repetir con contador	`for`	`for (let i=0; i<5; i++)`
Recorrer valores de un array / claves de un objeto	`for...of` / `for...in`	`for (const x of datos)`
Salir de un bucle / saltar una vuelta	`break` / `continue`	`if (encontrado) break`
Controlar o lanzar errores	`try / catch / finally` · `throw`	`throw Error("Fallo")`
Funciones y desestructuración		
Recoger muchos argumentos / expandir un array	rest `...` / spread `...`	`(...nums) => ...` · `Math.max(...nums)`
Extraer valores de un array u objeto	Desestructuración	`const [a, b] = datos` · `const {nombre} = persona`
Arrays		
Añadir / quitar al final	`push` / `pop`	`a.push(4)`
Añadir / quitar al principio	`unshift` / `shift`	`a.unshift(0)`
Insertar, borrar o sustituir en medio	`splice`	`a.splice(1, 1, 10)`
Copiar una parte sin tocar el original	`slice`	`a.slice(1, 3)`
Unir arrays	`concat` o spread	`a.concat(b)` · `[...a, ...b]`
¿Existe este valor?	`includes`	`a.includes(2)`
Posición de un valor / de lo que cumple una condición	`indexOf` / `findIndex`	`a.indexOf("b")`
Primer elemento que cumple / todos los que cumplen	`find` / `filter`	`a.find(n => n > 5)`
Transformar cada elemento	`map`	`a.map(n => n * 2)`
Hacer algo con cada elemento	`forEach`	`a.forEach(n => console.log(n))`
Sumar, contar o acumular todo	`reduce`	`a.reduce((t, n) => t + n, 0)`
¿Cumple alguno / cumplen todos?	`some` / `every`	`a.some(n => n > 5)`
Ordenar números / invertir	`sort((a,b) => a-b)` / `reverse`	`a.sort((x, y) => x - y)`
Texto <-> array	`split` / `join`	`"a,b".split(",")` · `["a","b"].join("-")`

11. Mini-test de práctica

Resuélvelo sin mirar las soluciones (están debajo, en la misma página). Las preguntas 1 a 12 piden predecir el resultado; la 13 y la 14 piden escribir código.

1	`let a=[1,2,3]; a.push(4); a.shift();` ¿Cuánto vale `a`?
2	`[5, 10, 1].sort()` ¿Qué devuelve?
3	`[1,2,3,4].filter(n => n % 2 === 0).map(n => n * 10)`
4	`[1,2,3,4].reduce((t, n) => t + n, 0)`
5	`let a=[1,2,3,4,5]; a.splice(1, 2, "x");` ¿Cuánto vale `a`?
6	`[1,2,3,4].slice(1, 3)`
7	`null ?? "N/A"` y `0 ?? "N/A"`
8	`typeof []` y `Array.isArray([])`
9	`const { nombre, edad = 18 } = { nombre: "Ana" };` ¿Valor de `nombre` y `edad`?
10	`[1,2,3].map(n => { n * 2 })`
11	`"a,b,c".split(",").reverse().join("-")`
12	`5 == "5"` y `5 === "5"`
13	ESCRIBE: una función flecha `sumaPares(arr)` que devuelva la suma de los números pares de un array (usa `filter` y `reduce`).
14	ESCRIBE: dado `usuarios = [{nombre:"Ana",edad:20},{nombre:"Luis",edad:15},{nombre:"Eva",edad:30}]`, obtén un array con los nombres de los mayores de edad.

Soluciones

1	`[2, 3, 4]` (push añade el 4 al final; shift quita el 1 del principio)
2	`[1, 10, 5]` (sin función ordena como texto)
3	`[20, 40]` (primero deja los pares [2,4] y luego multiplica)
4	`10`
5	`[1, "x", 4, 5]` (borra el 2 y el 3 y mete "x"; splice devuelve [2,3])
6	`[2, 3]` (el índice final 3 no se incluye)
7	`"N/A"` y `0` (el 0 no es null ni undefined)
8	`"object"` y `true`
9	`nombre = "Ana"`, `edad = 18` (valor predeterminado porque no llega edad)
10	`[undefined, undefined, undefined]` (con llaves falta el `return`)
11	`"c-b-a"`
12	`true` y `false`
13	`const sumaPares = arr => arr.filter(n => n % 2 === 0).reduce((t, n) => t + n, 0);` -> `sumaPares([1,2,3,4,6])` da 12
14	`usuarios.filter(u => u.edad >= 18).map(u => u.nombre)` -> `["Ana", "Eva"]`

Nota: esta guía reorganiza el contenido de tus dos guías de repaso (basadas en los PDF del profesor) y añade explicaciones, diagramas, trampas típicas y práctica. Los puntos marcados como «ampliación» o «ejemplo» son refuerzo; si algo no aparece en tus apuntes de clase, confírmalo con el material del profesor.